

Objektinis

Sugeneruota Doxygen 1.13.2

1 Aprašymas	1
1.1 Naudojimo instrukcijos	1
1.1.1 Duomenų įvestis	1
1.1.2 Duomenų išvestis	1
1.2 Versija 1.5	2
1.3 Bazinė ir išvestinė klasės	2
1.3.1 Zmogus bazinė klasė	2
1.3.2 Student išvestinė klasė	3
1.3.3 Testavimas	4
1.3.4 Rule of five:	4
1.3.5 Įvestis:	4
1.3.6 Išvestis:	4
1.3.7 Destruktorius:	4
1.3.8 Papildomi failai testavimui	4
1.4 Programos versijos	4
1.4.1 v.pradinė	4
1.4.2 v0.1	5
1.4.3 v0.2	5
1.4.4 v0.3	5
1.4.5 v0.4	5
1.4.6 v1.0prad	5
1.4.7 v1.0	5
1.4.8 v1.1	5
1.4.9 v1.2	5
1.4.10 v1.5	5
2 Hierarchijos Indeksas	7
2.1 Klasių hierarchija	7
3 Klasės Indeksas	9
3.1 Klasės	9
4 Failo Indeksas	11
4.1 Failai	11
5 Klasės Dokumentacija	13
5.1 Student Klasė	13
5.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	15
5.1.1.1 Student() [1/4]	15
5.1.1.2 Student() [2/4]	15
5.1.1.3 Student() [3/4]	16
5.1.1.4 Student() [4/4]	16
5.1.1.5 ~Student()	16
5.1.2 Metodų Dokumentacija	16

5.1.2.1 average()	16
5.1.2.2 calculateFinalMarks()	17
5.1.2.3 getAvgFinal()	17
5.1.2.4 getExamMark()	17
5.1.2.5 getHomeworkMarks()	18
5.1.2.6 getMedianFinal()	18
5.1.2.7 median()	18
5.1.2.8 operator=() [1/2]	18
5.1.2.9 operator=() [2/2]	19
5.1.2.10 print()	19
5.1.2.11 read()	19
5.1.2.12 readFromFile()	20
5.1.2.13 readInput()	20
5.1.2.14 readStudent()	21
5.1.2.15 setExamMark()	22
5.1.2.16 setMarks()	22
5.1.3 Atributų Dokumentacija	23
5.1.3.1 avgFinal_	23
5.1.3.2 examMark_	23
5.1.3.3 marks_	23
5.1.3.4 medianFinal_	23
5.2 TimeMeasurement Klasė	23
5.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	24
5.2.1.1 TimeMeasurement()	24
5.2.2 Metodų Dokumentacija	24
5.2.2.1 getActionName()	24
5.2.2.2 getTimeDifference()	24
5.2.2.3 start()	24
5.2.2.4 stop()	25
5.2.3 Atributų Dokumentacija	25
5.2.3.1 actionName	25
5.2.3.2 endTime	25
5.2.3.3 startTime	25
5.3 Zmogus Klasė	26
5.3.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	26
5.3.1.1 Zmogus() [1/2]	26
5.3.1.2 Zmogus() [2/2]	27
5.3.1.3 ~Zmogus()	27
5.3.2 Metodų Dokumentacija	27
5.3.2.1 getFirstName()	27
5.3.2.2 getLastName()	27
5.3.2.3 print()	28

5.3.2.4 read()	28
5.3.2.5 setFirstName()	28
5.3.2.6 setLastName()	29
5.3.3 Atributų Dokumentacija	29
5.3.3.1 firstName_	29
5.3.3.2 lastName_	29
6 Failo Dokumentacija	31
6.1 include/constants.h Failo Nuoroda	31
6.1.1 Kintamojo Dokumentacija	32
6.1.1.1 ADD_ANOTHER_MARK	32
6.1.1.2 ADD_ANOTHER_STUDENT	32
6.1.1.3 EMPTY_ARRAY_ERROR	32
6.1.1.4 ENTER_EXAM_MARK	32
6.1.1.5 ENTER_FILE_SIZE	33
6.1.1.6 ENTER_FIRST_NAME	33
6.1.1.7 ENTER_GROUP_TYPE	33
6.1.1.8 ENTER_LAST_NAME	33
6.1.1.9 ENTER_MARK	33
6.1.1.10 ENTER_MARK_IN_MIXED	33
6.1.1.11 ENTER_MARKS_COUNT	33
6.1.1.12 ENTER_PRINT_TYPE	33
6.1.1.13 ENTER_SORT_TYPE	33
6.1.1.14 FILE_OPEN_ERROR	34
6.1.1.15 FILE_READ_ERROR	34
6.1.1.16 FILE_READ_SUCCESS	34
6.1.1.17 FILE_WRITE_SUCCESS	34
6.1.1.18 INVALID_CHOICE	34
6.1.1.19 INVALID_EXAM_MARK_ERROR	34
6.1.1.20 INVALID_FILE_SIZE	34
6.1.1.21 INVALID_FIRST_NAME_ERROR	34
6.1.1.22 INVALID_GROUP_TYPE	34
6.1.1.23 INVALID_LAST_NAME_ERROR	34
6.1.1.24 INVALID_MARK_ERROR	35
6.1.1.25 INVALID_MARKS_COUNT	35
6.1.1.26 INVALID_MARKS_COUNT_ERROR	35
6.1.1.27 INVALID_MENU_CHOICE	35
6.1.1.28 INVALID_OUTPUT_TYPE_ERROR	35
6.1.1.29 INVALID_SIZE_ERROR	35
6.1.1.30 MENU_TEXT	35
6.1.1.31 PROGRAM_MENU_TEXT	35
6.1.1.32 TEST_MENU_TEXT	36

6.1.1.33 TEST_SECCCESS	36
6.2 constants.h	36
6.3 include/functions.h Failo Nuoroda	37
6.3.1 Funkcijos Dokumentacija	38
6.3.1.1 fileGenTest()	38
6.3.1.2 generateFile()	38
6.3.1.3 groupStudents()	39
6.3.1.4 handleProgramMenu()	39
6.3.1.5 handleTestMenu()	40
6.3.1.6 printToConsole()	40
6.3.1.7 printToFile()	41
6.3.1.8 programTest()	41
6.3.1.9 testRuleOfFive()	42
6.4 functions.h	43
6.5 include/mylib.h Failo Nuoroda	43
6.6 mylib.h	44
6.7 include/student.h Failo Nuoroda	45
6.7.1 Funkcijos Dokumentacija	46
6.7.1.1 sortStudents()	46
6.7.2 Kintamojo Dokumentacija	46
6.7.2.1 dstCount	46
6.8 student.h	46
6.9 include/timeMeasurement.h Failo Nuoroda	47
6.10 timeMeasurement.h	48
6.11 include/utlis.h Failo Nuoroda	48
6.11.1 Funkcijos Dokumentacija	49
6.11.1.1 GetExamMark()	49
6.11.1.2 getFileSize()	50
6.11.1.3 GetFirstName()	50
6.11.1.4 getGroupType()	51
6.11.1.5 GetHomeworkMarks()	51
6.11.1.6 GetLastName()	52
6.11.1.7 getMenuChoice()	52
6.11.1.8 getPrintType()	53
6.11.1.9 getProgramMenuChoice()	53
6.11.1.10 getRandomFirstName()	53
6.11.1.11 getRandomLastName()	54
6.11.1.12 getRandomMark()	54
6.11.1.13 getSortType()	54
6.11.1.14 getTestMenuChoice()	55
6.11.1.15 getYesNo()	55
6.11.1.16 isChoiceValid()	55

6.11.1.17 isMarkValid()	56
6.11.1.18 isNameValid()	56
6.11.1.19 printCount()	56
6.12 utils.h	57
6.13 include/zmogus.h Failo Nuoroda	57
6.13.1 Funkcijos Dokumentacija	58
6.13.1.1 operator<<()	58
6.13.1.2 operator>>()	59
6.14 zmogus.h	59
6.15 README.md Failo Nuoroda	59
6.16 src/functions.cpp Failo Nuoroda	59
6.16.1 Funkcijos Dokumentacija	60
6.16.1.1 fileGenTest()	60
6.16.1.2 generateFile()	61
6.16.1.3 groupStudents()	61
6.16.1.4 handleProgramMenu()	62
6.16.1.5 handleTestMenu()	62
6.16.1.6 printToConsole()	63
6.16.1.7 printToFile()	64
6.16.1.8 programTest()	64
6.16.1.9 testRuleOfFive()	65
6.17 src/main.cpp Failo Nuoroda	65
6.17.1 Funkcijos Dokumentacija	66
6.17.1.1 main()	66
6.18 src/student.cpp Failo Nuoroda	67
6.18.1 Funkcijos Dokumentacija	67
6.18.1.1 sortStudents()	67
6.18.2 Kintamojo Dokumentacija	67
6.18.2.1 dstCount	67
6.19 src/timeMeasurement.cpp Failo Nuoroda	68
6.20 src/utils.cpp Failo Nuoroda	68
6.20.1 Funkcijos Dokumentacija	69
6.20.1.1 GetExamMark()	69
6.20.1.2 getFileSize()	70
6.20.1.3 GetFirstName()	70
6.20.1.4 getGroupType()	71
6.20.1.5 GetHomeworkMarks()	71
6.20.1.6 GetLastName()	72
6.20.1.7 getMenuChoice()	72
6.20.1.8 getPrintType()	73
6.20.1.9 getProgramMenuChoice()	73
6.20.1.10 getRandomFirstName()	73

6.20.1.11 getRandomLastName()	74
6.20.1.12 getRandomMark()	74
6.20.1.13 getSortType()	74
6.20.1.14 getTestMenuChoice()	75
6.20.1.15 getYesNo()	75
6.20.1.16 isChoiceValid()	75
6.20.1.17 isMarkValid()	76
6.20.1.18 isNameValid()	76
6.20.1.19 mt()	76
6.20.1.20 printCount()	77
6.20.2 Kintamojo Dokumentacija	77
6.20.2.1 rd	77

skyrius 1

Aprašymas

Programa, skirta apdoroti studentų duomenis. Programos versijose analizuojamas bei tobulinamas programos našumas ir efektyvumas.

1.1 Naudojimo instrukcijos

Kad paleisti programą, turite atlikti šiuos veiksmus:

1. Įeikite į norimos versijos katalogą
2. Paleiskite `run.bat` failą
`run.bat` failas atliks šiuos veiksmus:
 - Sukurs build katalogą
 - Paleis CMake, kad sugeneruotų `Makefile`
 - Sukompiliuos projektą naudodamas `make` komandą
 - Paleis sukompiliuotą programą terminale

Reikalavimai

- Operacinė sistema: Windows 10 x64-bit arba naujesnė versija
- Įdiegta CMake (3.25 arba naujesnė versija)
- Kompiliatorius: g++ (su C++11 arba naujesne versija)

1.1.1 Duomenų įvestis

Tipas	Aprašymas
Rankinis	Vartotojas suveda duomenis ranka tiesiai į konsolę
Automatinis	Naudojamas testavimui su <code>istream</code>
Iš failo	Skaitomi duomenys iš failo naudojant <code>ifstream</code> (v1.2 teste numatytas vienas dydis - 3, kitur galima rinktis iš 5 dydžių)

1.1.2 Duomenų išvestis

Tipas	Aprašymas
Į ekraną	Duomenys atvaizduojami konsolėje per <code>cout <<</code>
Į failą	Duomenys saugomi faile per <code>ostream</code>

1.2 Versija 1.5

1.3 Bazinė ir išvestinė klasės

Sukurta nauja bazinė klasė `Zmogus`, klasė `Student` paversta į derived. Faile `functions` sukurtas naujas testas `testZmogusClass()`, kuris patikrina, ar `Zmogus` klasė yra abstrakti. Panaudotas testas iš v1.2 `testRuleOfFive()`, kuris patikrina visus metodus.

1.3.1 Zmogus bazinė klasė

Protected atributai

- `firstName_` - studento vardas
- `lastName_` - studento pavardė

Konstruktoriai

- `Zmogus()` – default konstruktorius
- `Zmogus(const std::string& firstName, const std::string& lastName)` – konstruktorius su parametrais
- `virtual ~Zmogus() = 0` – virtualus abstrakčios klasės destruktorius

Virtualios funkcijos

- `virtual void read(std::istream& is) = 0` – input'ui
- `virtual void print(std::ostream& os) const = 0` – output'ui

Getteriai ir setteriai

- `string getFirstName() const` – grąžina vardą
- `string getLastName() const` – grąžina pavardę
- `void setFirstName(const std::string& firstName)` – nustato vardą
- `void setLastName(const std::string& lastName)` – nustato pavardę

Globalūs operatoriai

- `inline std::istream& operator>>(std::istream& is, Zmogus& zmogus)`
- `inline std::ostream& operator<<(std::ostream& os, const Zmogus& zmogus)`

1.3.2 Student išvestinė klasė

Privatūs atributai

- `std::vector<int> marks_` - pažymiai
- `int examMark_` - egzamino pažymys
- `float avgFinal_`; - galutinis pažymys pagal vidurkį
- `float medianFinal_`; - galutinis pažymys pagal medianą

Privatūs metodai

- `float average(const std::vector<int>& marks) const` - apskaičiuoja galutinį pažymį pagal vidurkį
- `float median(const std::vector<int>& marks) const` - apskaičiuoja galutinį pažymį pagal medianą

Konstruktoriai

- `Student()` – default konstruktorius
- `Student(std::istream& is)` – konstruktorius su input stream'u
- `Student::Student(const Student& other)` – kopijavimo konstruktorius
- `Student::Student(Student&& other) noexcept` - perkėlimo konstruktorius

Persidengimo operatoriai

- `Student& Student::operator=(const Student& other)` - kopijavimo operatorius
- `Student& Student::operator=(Student&& other) noexcept` - perkėlimo operatorius

Getteriai ir setteriai

- `const std::vector<int>& getHomeworkMarks()` const - grąžina pažymius
- `int getExamMark()` const – grąžina egzamino pažymį
- `float getAvgFinal()` const – grąžina galutinį pažymį pagal vidurkį
- `float getMedianFinal()` const – grąžina galutinį pažymį pagal medianą
- `void setMarks(std::vector<int> marks)` – nustato pažymius
- `void setExamMark(int examMark)` - nustato galutinį pažymį

Įgyvendintos virtualios funkcijos

- `void Student::read(std::istream& is)` – input'ui
- `void Student::print(std::ostream& os) const` – output'ui

Kiti metodai

- `std::istream& readStudent(std::istream& is)` - input'ui
- `void calculateFinalMarks();` - iškviečia 2 galutinio pažymio skaičiavimo funkcijas (vid. ir med.)
- `static void readInput(std::vector<Student>& students, char menuChoice);`
- skaito studento duomenis įvestus konsolėje
- `static void readFromFile(std::vector<Student>& students, int fileSize);`
- skaito studento duomenis iš failo

1.3.3 Testavimas

1. Funkcija `testZmogusClass()` Tikrina ar klasė yra abstrakti. Norint pratestuoti, reikia atkomentuoti šią funkciją.

Kadangi `Zmogus` `zmogus` nėra kompiliuojamas, klasė yra abstrakti.

2. Funkcija `testRuleOfFive()` Tikrina:

1. Default konstruktorius
2. Kopijavimo konstruktorius
3. Kopijavimo priskyrimo operatorius
4. Perkėlimo konstruktorius
5. Perkėlimo priskyrimo operatorius
6. Įvesties operatoriai
 - 6.1 Įvestį iš konsolės
 - 6.2 Įvestį iš failo
7. Išvesties operatorių
8. Nuskaitymą iš failo
9. Išvedimą į failą
10. Išvedimą į konsolę
11. Destruktorių

1.3.4 Rule of five:

1.3.5 Įvestis:

1.3.6 Išvestis:

1.3.7 Destruktorius:

1.3.8 Papildomi failai testavimui

`testas1.txt` - 6.2 testui `studentai3.txt` - 8 testui `testas2.txt` - 9 testui

1.4 Programos versijos

1.4.1 v.pradinė

C++ programa, skirta studentų pažymiams apskaičiuoti, naudojant vidurkio ir medianos metodus. Vartotojai įveda studentų vardus, namų darbų pažymius ir egzaminų rezultatus, o programa pateikia suformatuotus rezultatus. `Makefile` automatizuoja kompiliavimą, o `gitignore` pašalina nereikalingus failus iš „Git“ sekimo.

1.4.2 v0.1

Optimizuoti failai ir pataisytos galutinio pažymio apskaičiavimo funkcijos. Įgyvendinti du skirtingi vykdomieji failai: vienas su vektorių masyvais, kitas – su dinaminiais C masyvais pažymiams. Pridėta `menu` funkcija, leidžianti atsitiiktinai generuoti pasirinktus duomenis.

1.4.3 v0.2

Prie `menu` pridėta failų skaitymo parinktis, rūšiavimo parinktys (pagal vardą, pavardę ir galutinį pažymį) ir parinktis pasirinkti, ar spausdinti rezultatą į monitorių, ar į failą. Taip pat pridėti `TimeMeasurement` failai, skirti sekti laiką ir apskaičiuoti vidutinius testavimo laikus.

1.4.4 v0.3

Pridėtas išimčių tvarkymas funkcijoms `readFromFile`, `output` ir `sortStudents`. Sukurti aplankai ir atnaujintas `makefile`.

1.4.5 v0.4

Pridėta failų generavimo funkcija su pasirinktu dydžiu, studentų grupavimas į dvi grupes pagal pasirinktą galutinio pažymio tipą ir dvi laiko matavimo testavimo funkcijos. Pakeista `timeMeasurement` funkcija, kad testų rezultatai būtų spausdinami į failą, o ne į konsolę. Atnaujintas `README`, įtraukiant laiko matavimo testų rezultatus. Optimizuotos kelios funkcijos, siekiant geresnio našumo.

1.4.6 v1.0prad

Pridėta nauja laiko testavimo funkcija ir versija su 3 skirtingais konteineriais: `vector`, `deque` ir `list`. Atliktas laiko tyrimas, o rezultatai pateikti faile `README`.

1.4.7 v1.0

Pridėti testavimai su trimis skirtingomis strategijomis studentų rūšiavimo į grupes funkcijai. Optimizuotas geriausias rūšiavimas (`vector` konteinerio su pirma strategija). Atliktas spartos bei atminties naudojimo tyrimas, o rezultatai pateikti faile `README`. Pridėtas programos diegimo bei paleidimo `CMake` failas bei `README` aprašyta įdiegimo instrukcija.

1.4.8 v1.1

`Vector` konteinerio versijoje duomenų struktūra pakeista iš `struct` į `class`. Atlikti programos spartos bei SSD naudojimo testai lyginant `struct` ir `class` su skirtingomis optimizavimo vėliavėlėmis: `O1`, `O2` ir `O3`. Tyrimo rezultatai aprašyti `README` faile.

1.4.9 v1.2

Pašalinti `deque` bei `list` versijų katalogai. Pritaikyta Rule of Five ir persidengimo operatoriai. Atliktas testas patikrinantis `Student` klasės metodų veikimą.

1.4.10 v1.5

Sukurta nauja bazinė klasė `Zmogus`. `Student` klasė paversta į `derived`. Patikrinta, ar nauji metodai praeina testus iš versijos 1.2

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

TimeMeasurement	23
Zmogus	26
Student	13

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Student	13
TimeMeasurement	23
Zmogus	26

skyrius 4

Failo Indeksas

4.1 Failai

Visų failų sąrašas su trumpais aprašymais:

include/constants.h	31
include/functions.h	37
include/mylib.h	43
include/student.h	45
include/timeMeasurement.h	47
include/utils.h	48
include/zmogus.h	57
src/functions.cpp	59
src/main.cpp	65
src/student.cpp	67
src/timeMeasurement.cpp	68
src/utils.cpp	68

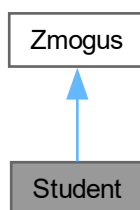
skyrius 5

Klasės Dokumentacija

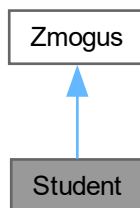
5.1 Student Klasė

```
#include <student.h>
```

Paveldimumo diagrama Student:



Bendradarbiavimo diagrama Student:



Vieši Metodai

- `Student ()`
- `Student (std::istream &is)`
- `Student (const Student &other)`
- `Student & operator= (const Student &other)`
- `Student (Student &&other) noexcept`
- `Student & operator= (Student &&other) noexcept`
- `~Student ()`
- `const std::vector< int > & getHomeworkMarks () const`
- `int getExamMark () const`
- `float getAvgFinal () const`
- `float getMedianFinal () const`
- `void setMarks (std::vector< int > marks)`
- `void setExamMark (int examMark)`
- `void print (std::ostream &os) const override`
- `void read (std::istream &is) override`
- `std::istream & readStudent (std::istream &is)`
- `void calculateFinalMarks ()`

Vieši Metodai inherited from `Zmogus`

- `Zmogus ()`
- `Zmogus (const std::string &firstName, const std::string &lastName)`
- `virtual ~Zmogus ()=default`
- `std::string getFirstName () const`
- `std::string getLastName () const`
- `void setFirstName (const std::string &firstName)`
- `void setLastName (const std::string &lastName)`

Statiniai Vieši Metodai

- `static void readInput (std::vector< Student > &students, char menuChoice)`
- `static void readFromFile (std::vector< Student > &students, int fileSize)`

Privatatūs Metodai

- `float average (const std::vector< int > &marks) const`
- `float median (const std::vector< int > &marks) const`

Privatūs Atributai

- `std::vector< int > marks_`
- `int examMark_`
- `float avgFinal_`
- `float medianFinal_`

Additional Inherited Members

Apsaugoti Atributai inherited from Zmogus

- std::string `firstName_`
- std::string `lastName_`

5.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

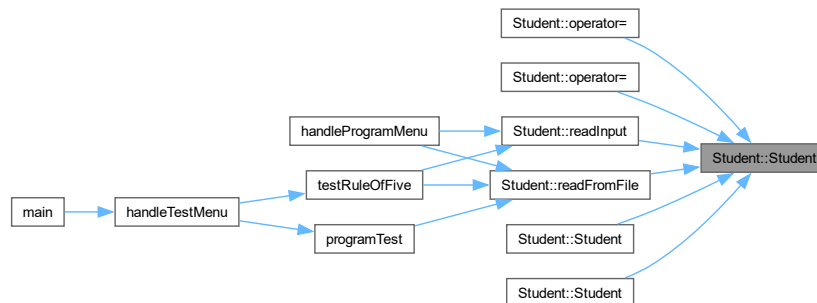
5.1.1.1 Student() [1/4]

```
Student::Student ()
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



5.1.1.2 Student() [2/4]

```
Student::Student (
    std::istream & is)
```

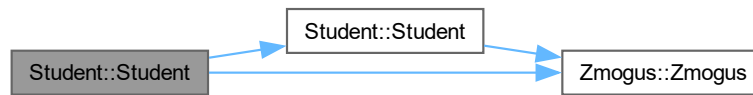
Funkcijos kvietimo grafas:



5.1.1.3 Student() [3/4]

```
Student::Student (
    const Student & other)
```

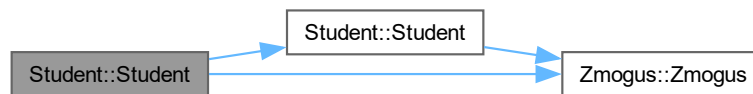
Funkcijos kvietimo grafas:



5.1.1.4 Student() [4/4]

```
Student::Student (
    Student && other) [noexcept]
```

Funkcijos kvietimo grafas:



5.1.1.5 ~Student()

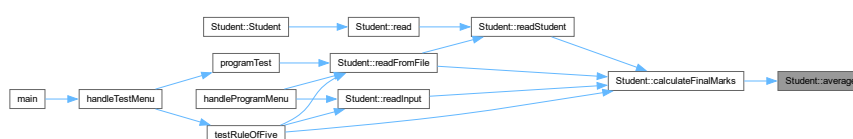
```
Student::~~Student ()
```

5.1.2 Metodų Dokumentacija

5.1.2.1 average()

```
float Student::average (
    const std::vector< int > & marks) const [private]
```

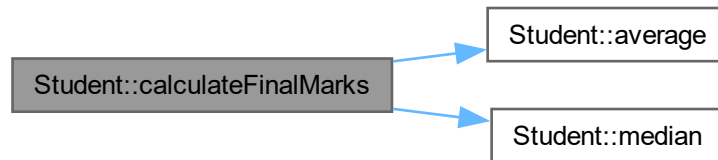
Here is the caller graph for this function:



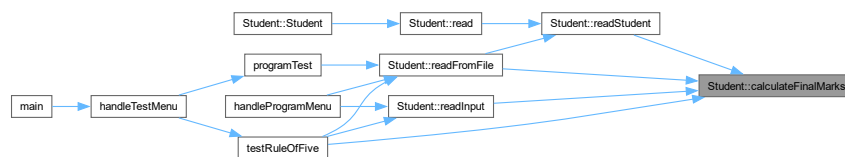
5.1.2.2 calculateFinalMarks()

```
void Student::calculateFinalMarks ()
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



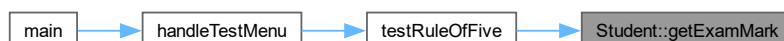
5.1.2.3 getAvgFinal()

```
float Student::getAvgFinal () const [inline]
```

5.1.2.4 getExamMark()

```
int Student::getExamMark () const [inline]
```

Here is the caller graph for this function:



5.1.2.5 getHomeworkMarks()

```
const std::vector< int > & Student::getHomeworkMarks () const [inline]
```

Here is the caller graph for this function:



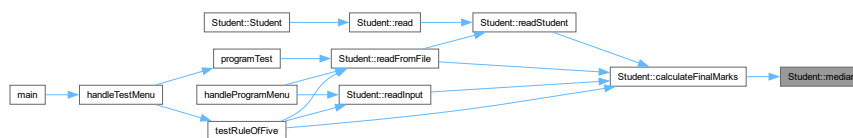
5.1.2.6 getMedianFinal()

```
float Student::getMedianFinal () const [inline]
```

5.1.2.7 median()

```
float Student::median (
    const std::vector< int > & marks) const [private]
```

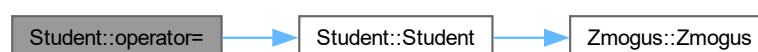
Here is the caller graph for this function:



5.1.2.8 operator=() [1/2]

```
Student & Student::operator= (
    const Student & other)
```

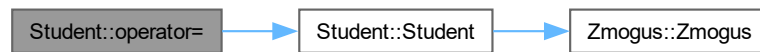
Funkcijos kvietimo grafas:



5.1.2.9 operator=() [2/2]

```
Student & Student::operator= (
    Student && other) [noexcept]
```

Funkcijos kvietimo grafas:



5.1.2.10 print()

```
void Student::print (
    std::ostream & os) const [override], [virtual]
```

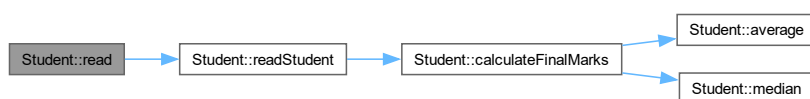
Realizuoja [Zmogus](#).

5.1.2.11 read()

```
void Student::read (
    std::istream & is) [override], [virtual]
```

Realizuoja [Zmogus](#).

Funkcijos kvietimo grafas:



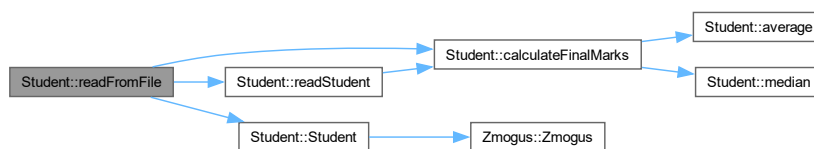
Here is the caller graph for this function:



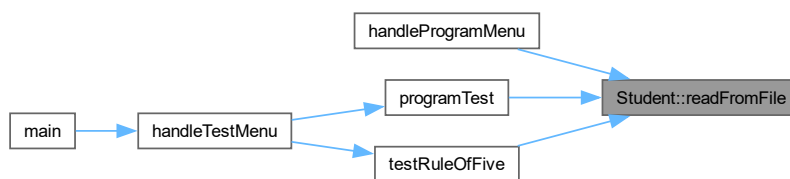
5.1.2.12 readFromFile()

```
void Student::readFromFile (
    std::vector< Student > & students,
    int fileSize) [static]
```

Funkcijos kvietimo grafas:



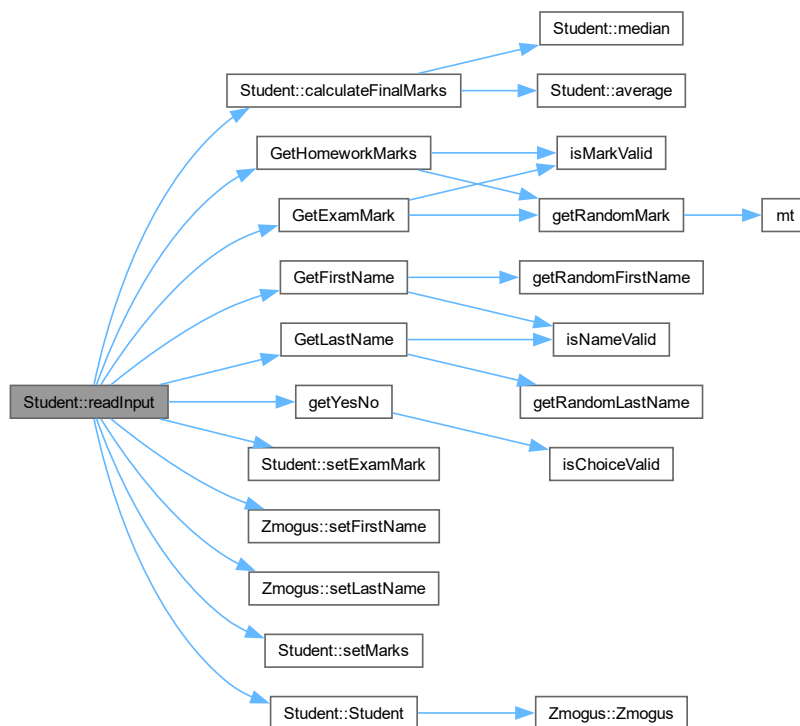
Here is the caller graph for this function:



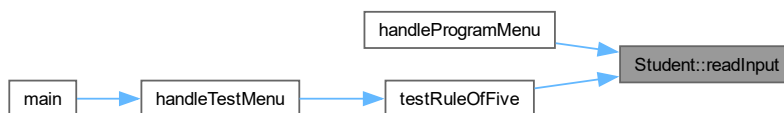
5.1.2.13 readInput()

```
void Student::readInput (
    std::vector< Student > & students,
    char menuChoice) [static]
```

Funkcijos kvietimo grafas:



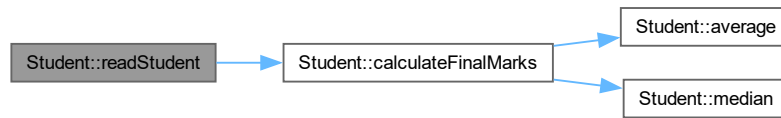
Here is the caller graph for this function:



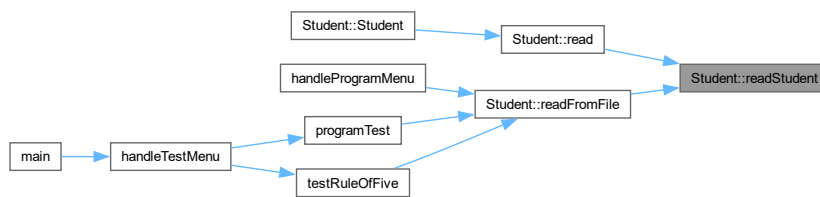
5.1.2.14 readStudent()

```
std::istream & Student::readStudent (
    std::istream & is)
```

Funkcijas kvietimo grafas:



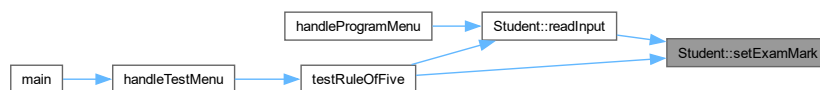
Here is the caller graph for this function:



5.1.2.15 `setExamMark()`

```
void Student::setExamMark (
    int examMark) [inline]
```

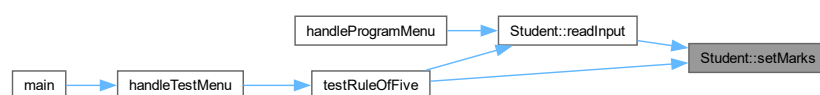
Here is the caller graph for this function:



5.1.2.16 `setMarks()`

```
void Student::setMarks (
    std::vector< int > marks) [inline]
```

Here is the caller graph for this function:



5.1.3 Atributų Dokumentacija

5.1.3.1 avgFinal_

```
float Student::avgFinal_ [private]
```

5.1.3.2 examMark_

```
int Student::examMark_ [private]
```

5.1.3.3 marks_

```
std::vector<int> Student::marks_ [private]
```

5.1.3.4 medianFinal_

```
float Student::medianFinal_ [private]
```

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [include/student.h](#)
- [src/student.cpp](#)

5.2 TimeMeasurement Klasė

```
#include <timeMeasurement.h>
```

Vieši Metodai

- [TimeMeasurement](#) (const std::string &name)
- void [start](#) ()
- void [stop](#) (std::ofstream &runTimeResults)
- double [getTimeDifference](#) ()
- std::string [getActionName](#) () const

Privatūs Atributai

- std::chrono::high_resolution_clock::time_point [startTime](#)
- std::chrono::high_resolution_clock::time_point [endTime](#)
- std::string [actionName](#)

5.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.2.1.1 TimeMeasurement()

```
TimeMeasurement::TimeMeasurement (  
    const std::string & name)
```

5.2.2 Metodų Dokumentacija

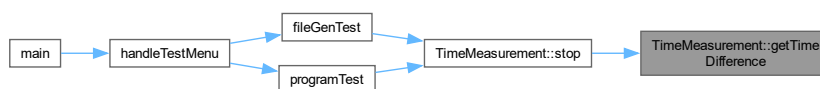
5.2.2.1 getActionName()

```
std::string TimeMeasurement::getActionName () const [inline]
```

5.2.2.2 getTimeDifference()

```
double TimeMeasurement::getTimeDifference ()
```

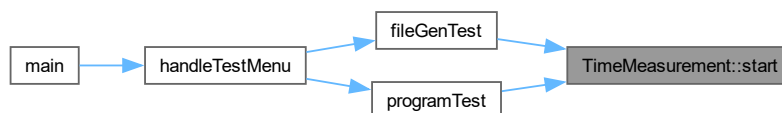
Here is the caller graph for this function:



5.2.2.3 start()

```
void TimeMeasurement::start ()
```

Here is the caller graph for this function:



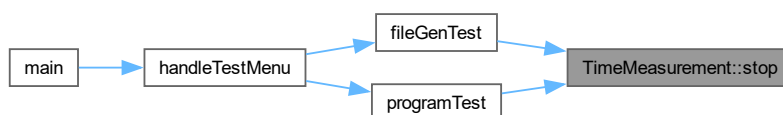
5.2.2.4 stop()

```
void TimeMeasurement::stop (
    std::ofstream & runTimeResults)
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



5.2.3 Atributų Dokumentacija

5.2.3.1 actionName

```
std::string TimeMeasurement::actionName [private]
```

5.2.3.2 endTime

```
std::chrono::high_resolution_clock::time_point TimeMeasurement::endTime [private]
```

5.2.3.3 startTime

```
std::chrono::high_resolution_clock::time_point TimeMeasurement::startTime [private]
```

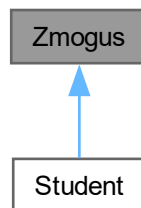
Dokumentacija šiai klasei sugeneruota iš šių failų:

- [include/timeMeasurement.h](#)
- [src/timeMeasurement.cpp](#)

5.3 Zmogus Klasė

```
#include <zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &firstName, const std::string &lastName)
- virtual [~Zmogus](#) ()=default
- virtual void [read](#) (std::istream &is)=0
- virtual void [print](#) (std::ostream &os) const =0
- std::string [getFirstName](#) () const
- std::string [getLastName](#) () const
- void [setFirstName](#) (const std::string &firstName)
- void [setLastName](#) (const std::string &lastName)

Apsaugoti Atributai

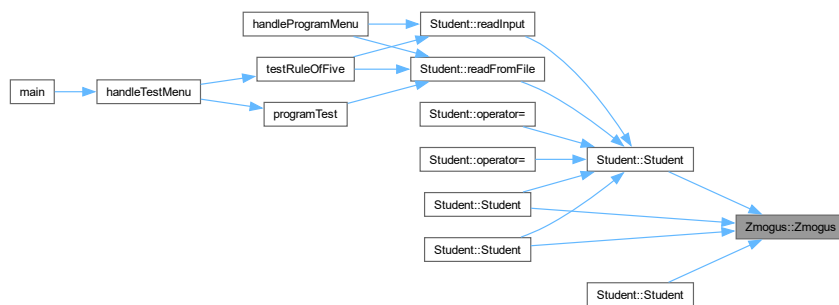
- std::string [firstName_](#)
- std::string [lastName_](#)

5.3.1 Konstruktorius ir Destruktorius Dokumentacija

5.3.1.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

Here is the caller graph for this function:



5.3.1.2 Zmogus() [2/2]

```

Zmogus::Zmogus (
    const std::string & firstName,
    const std::string & lastName) [inline]
  
```

5.3.1.3 ~Zmogus()

```

virtual Zmogus::~Zmogus () [virtual], [default]
  
```

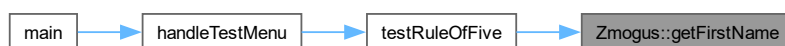
5.3.2 Metodų Dokumentacija

5.3.2.1 getFirstName()

```

std::string Zmogus::getFirstName () const [inline]
  
```

Here is the caller graph for this function:

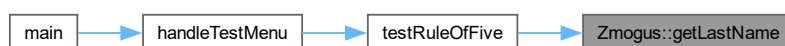


5.3.2.2 getLastName()

```

std::string Zmogus::getLastName () const [inline]
  
```

Here is the caller graph for this function:



5.3.2.3 print()

```
virtual void Zmogus::print (  
    std::ostream & os) const [pure virtual]
```

Realizuota [Student](#).

Here is the caller graph for this function:



5.3.2.4 read()

```
virtual void Zmogus::read (  
    std::istream & is) [pure virtual]
```

Realizuota [Student](#).

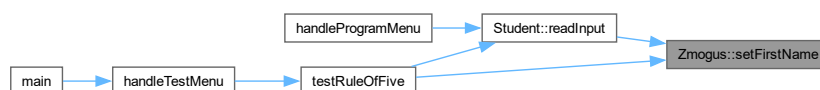
Here is the caller graph for this function:



5.3.2.5 setFirstName()

```
void Zmogus::setFirstName (  
    const std::string & firstName) [inline]
```

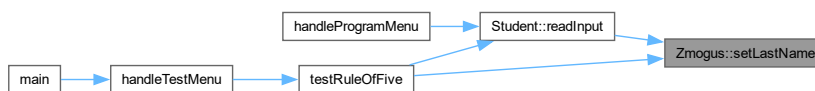
Here is the caller graph for this function:



5.3.2.6 setLastName()

```
void Zmogus::setLastName (
    const std::string & lastName) [inline]
```

Here is the caller graph for this function:



5.3.3 Atributų Dokumentacija

5.3.3.1 firstName_

```
std::string Zmogus::firstName_ [protected]
```

5.3.3.2 lastName_

```
std::string Zmogus::lastName_ [protected]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [include/zmogus.h](#)

skyrius 6

Failo Dokumentacija

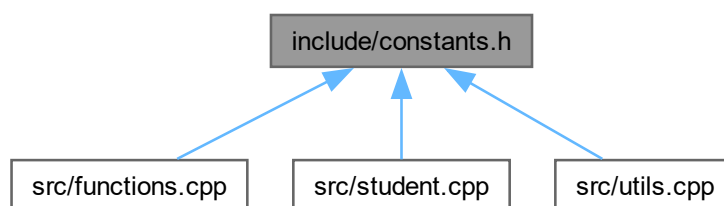
6.1 include/constants.h Failo Nuoroda

```
#include "mylib.h"
```

Įtraukimo priklausomybių diagrama constants.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Kintamieji

- const string ENTER_FIRST_NAME = "Įveskite vardą: "
- const string ENTER_LAST_NAME = "Įveskite pavardę: "
- const string ENTER_MARK = "Įveskite namų darbų pažymį (norint sustoti, įveskite -1): "
- const string ENTER_MARK_IN_MIXED = "Įveskite namų darbų pažymį: "
- const string ENTER_EXAM_MARK = "Įveskite egzamino pažymį: "
- const string ENTER_GROUP_TYPE = "Pasirinkite grupavimo tipą (1 - vidurkis / 2 - mediana): "

- const string ENTER_SORT_TYPE = "Pasirinkite, pagal ką rūšiuoti studentus (1 - vardą / 2 - pavardę / 3 - galutinį balą (vid.) / 4 - galutinį balą (med.)):"
- const string ENTER_PRINT_TYPE = "Pasirinkite, kur išvesti duomenis (1 - ekraną / 2 - failą): "
- const string ENTER_MARKS_COUNT = "Įveskite namų darbų pažymių skaičių: "
- const string ADD_ANOTHER_MARK = "Ar norite įvesti dar vieną namų darbą? (taip/ne): "
- const string ADD_ANOTHER_STUDENT = "Ar norite įvesti dar vieną studentą? (taip/ne): "
- constexpr char MENU_TEXT []
- constexpr char TEST_MENU_TEXT []
- constexpr char PROGRAM_MENU_TEXT []
- constexpr char ENTER_FILE_SIZE []
- const string INVALID_FIRST_NAME_ERROR = "\nKlaida: įvestas netinkamas vardas! "
- const string INVALID_LAST_NAME_ERROR = "\nKlaida: įvesta netinkama pavardė! "
- const string INVALID_MARK_ERROR = "\nKlaida: pažymys turi būti sveikasis skaičius nuo 1 iki 10! "
- const string INVALID_EXAM_MARK_ERROR = "\nKlaida: egzamino pažymys turi būti sveikasis skaičius nuo 1 iki 10! "
- const string INVALID_MARKS_COUNT_ERROR = "\nKlaida: pažymių skaičius turi būti sveikasis skaičius ir mažesnis nei 10! "
- const string INVALID_MENU_CHOICE = "\nKlaida: netinkamas meniu pasirinkimas! "
- const string INVALID_CHOICE = "\nKlaida: netinkamas pasirinkimas! "
- const string INVALID_SIZE_ERROR = "\nKlaida: dydis turi būti sveikasis skaičius! "
- const string INVALID_MARKS_COUNT = "\nKlaida: turite įvesti bent vieną pažymį! "
- const string INVALID_OUTPUT_TYPE_ERROR = "\nKlaida: netinkamas išvedimo tipas! "
- const string FILE_OPEN_ERROR = "\nKlaida: nepavyko atidaryti failo! "
- const string FILE_READ_ERROR = "\nKlaida: nepavyko nuskaityti failo! "
- const string EMPTY_ARRAY_ERROR = "\nKlaida: pažymių masyvas tuščias! "
- const string INVALID_GROUP_TYPE = "\nKlaida: netinkamas grupavimo tipas! "
- const string INVALID_FILE_SIZE = "\nKlaida: netinkamas failo dydis! "
- const string FILE_READ_SUCCESS = "\nDuomenys nuskaityti sėkmingai! "
- const string FILE_WRITE_SUCCESS = "\nDuomenys įrašyti sėkmingai! "
- const string TEST_SECEESS = "\nLaiko tyrimo rezultatai įrašyti į failą: "

6.1.1 Kintamojo Dokumentacija

6.1.1.1 ADD_ANOTHER_MARK

```
const string ADD_ANOTHER_MARK = "Ar norite įvesti dar vieną namų darbą? (taip/ne): "
```

6.1.1.2 ADD_ANOTHER_STUDENT

```
const string ADD_ANOTHER_STUDENT = "Ar norite įvesti dar vieną studentą? (taip/ne): "
```

6.1.1.3 EMPTY_ARRAY_ERROR

```
const string EMPTY_ARRAY_ERROR = "\nKlaida: pažymių masyvas tuščias! "
```

6.1.1.4 ENTER_EXAM_MARK

```
const string ENTER_EXAM_MARK = "Įveskite egzamino pažymį: "
```


6.1.1.5 ENTER_FILE_SIZE

```
char ENTER_FILE_SIZE[] [constexpr]
```

Pradinė reikšmė:

```
=  
    "\nPasirinkite failo dydį:\n"  
    "1 - 1 000\n"  
    "2 - 10 000\n"  
    "3 - 100 000\n"  
    "4 - 1 000 000\n"  
    "5 - 10 000 000\n"
```

6.1.1.6 ENTER_FIRST_NAME

```
const string ENTER_FIRST_NAME = "Įveskite vardą: "
```

6.1.1.7 ENTER_GROUP_TYPE

```
const string ENTER_GROUP_TYPE = "Pasirinkite grupavimo tipą (1 - vidurkis / 2 - mediana): "
```

6.1.1.8 ENTER_LAST_NAME

```
const string ENTER_LAST_NAME = "Įveskite pavardę: "
```

6.1.1.9 ENTER_MARK

```
const string ENTER_MARK = "Įveskite namų darbų pažymį (norint sustoti, įveskite -1): "
```

6.1.1.10 ENTER_MARK_IN_MIXED

```
const string ENTER_MARK_IN_MIXED = "Įveskite namų darbų pažymį: "
```

6.1.1.11 ENTER_MARKS_COUNT

```
const string ENTER_MARKS_COUNT = "Įveskite namų darbų pažymių skaičių: "
```

6.1.1.12 ENTER_PRINT_TYPE

```
const string ENTER_PRINT_TYPE = "Pasirinkite, kur išvesti duomenis (1 - ekraną / 2 - failą): "  
"
```

6.1.1.13 ENTER_SORT_TYPE

```
const string ENTER_SORT_TYPE = "Pasirinkite, pagal ką rūšiuoti studentus (1 - vardą / 2 -  
pavardę / 3 - galutinį balą (vid.) / 4 - galutinį balą (med.))): "
```

6.1.1.14 FILE_OPEN_ERROR

```
const string FILE_OPEN_ERROR = "\nKlaida: nepavyko atidaryti failo! "
```

6.1.1.15 FILE_READ_ERROR

```
const string FILE_READ_ERROR = "\nKlaida: nepavyko nuskaityti failo! "
```

6.1.1.16 FILE_READ_SUCCESS

```
const string FILE_READ_SUCCESS = "\nDuomenys nuskaityti sėkmingai! "
```

6.1.1.17 FILE_WRITE_SUCCESS

```
const string FILE_WRITE_SUCCESS = "\nDuomenys įrašyti sėkmingai! "
```

6.1.1.18 INVALID_CHOICE

```
const string INVALID_CHOICE = "\nKlaida: netinkamas pasirinkimas! "
```

6.1.1.19 INVALID_EXAM_MARK_ERROR

```
const string INVALID_EXAM_MARK_ERROR = "\nKlaida: egzamino pažymys turi būti sveikasis skaičius  
nuo 1 iki 10! "
```

6.1.1.20 INVALID_FILE_SIZE

```
const string INVALID_FILE_SIZE = "\nKlaida: netinkamas failo dydis! "
```

6.1.1.21 INVALID_FIRST_NAME_ERROR

```
const string INVALID_FIRST_NAME_ERROR = "\nKlaida: įvestas netinkamas vardas! "
```

6.1.1.22 INVALID_GROUP_TYPE

```
const string INVALID_GROUP_TYPE = "\nKlaida: netinkamas grupavimo tipas! "
```

6.1.1.23 INVALID_LAST_NAME_ERROR

```
const string INVALID_LAST_NAME_ERROR = "\nKlaida: įvesta netinkama pavardė! "
```

6.1.1.24 INVALID_MARK_ERROR

```
const string INVALID_MARK_ERROR = "\nKlaida: pažymys turi būti sveikasis skaičius nuo 1 iki 10! "
```

6.1.1.25 INVALID_MARKS_COUNT

```
const string INVALID_MARKS_COUNT = "\nKlaida: turite įvesti bent vieną pažymį! "
```

6.1.1.26 INVALID_MARKS_COUNT_ERROR

```
const string INVALID_MARKS_COUNT_ERROR = "\nKlaida: pažymių skaičius turi būti sveikasis skaičius ir mažesnis nei 10! "
```

6.1.1.27 INVALID_MENU_CHOICE

```
const string INVALID_MENU_CHOICE = "\nKlaida: netinkamas meniu pasirinkimas! "
```

6.1.1.28 INVALID_OUTPUT_TYPE_ERROR

```
const string INVALID_OUTPUT_TYPE_ERROR = "\nKlaida: netinkamas išvedimo tipas! "
```

6.1.1.29 INVALID_SIZE_ERROR

```
const string INVALID_SIZE_ERROR = "\nKlaida: dydis turi būti sveikasis skaičius! "
```

6.1.1.30 MENU_TEXT

```
char MENU_TEXT[] [constexpr]
```

Pradinė reikšmė:

```
=
"\nPasirinkite, ką norite daryti:\n"
"1 - Vykdyti programą\n"
"2 - Atlikti tyrimą\n"
"0 - Nieko nenoriu daryti\n"
```

6.1.1.31 PROGRAM_MENU_TEXT

```
char PROGRAM_MENU_TEXT[] [constexpr]
```

Pradinė reikšmė:

```
=
"\nPasirinkite, ką norite daryti:\n"
"1 - Įvesti viską rankiniu būdu\n"
"2 - Generuoti atsitiktinius pažymius (vardus įvesti ranka)\n"
"3 - Generuoti atsitiktinius vardus ir pažymius\n"
"4 - Skaityti iš failo\n"
"5 - Užbaigti programą\n"
```

6.1.1.32 TEST_MENU_TEXT

```
char TEST_MENU_TEXT[] [constexpr]
```

Pradinė reikšmė:

```
=
"\nPasirinkite tyrimą:\n"
"1 - failu generavimas\n"
"2 - programos veikimo laikas\n"
"3 - Rule of Five testavimas\n"
"0 - Užbaigti programą\n"
```

6.1.1.33 TEST_SECESS

```
const string TEST_SECESS = "\nLaiko tyrimo rezultatai įrašyti į failą: "
```

6.2 constants.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002 #include "mylib.h"
00003
00004 // user prompts
00005 const string ENTER_FIRST_NAME = "Įveskite vardą: ";
00006 const string ENTER_LAST_NAME = "Įveskite pavardę: ";
00007 const string ENTER_MARK = "Įveskite namų darbų pažymį (norint sustoti, įveskite -1): ";
00008 const string ENTER_MARK_IN_MIXED = "Įveskite namų darbų pažymį: ";
00009 const string ENTER_EXAM_MARK = "Įveskite egzamino pažymį: ";
00010 const string ENTER_GROUP_TYPE = "Pasirinkite grupavimo tipą (1 - vidurkis / 2 - mediana): ";
00011 const string ENTER_SORT_TYPE = "Pasirinkite, pagal ką rūšiuoti studentus (1 - vardą / 2 - pavardę / 3
- galutinį balą (vid.) / 4 - galutinį balą (med.)): ";
00012 const string ENTER_PRINT_TYPE = "Pasirinkite, kur išvesti duomenis (1 - ekraną / 2 - failą): ";
00013 const string ENTER_MARKS_COUNT = "Įveskite namų darbų pažymių skaičių: ";
00014 const string ADD_ANOTHER_MARK = "Ar norite įvesti dar vieną namų darbą? (taip/ne): ";
00015 const string ADD_ANOTHER_STUDENT = "Ar norite įvesti dar vieną studentą? (taip/ne): ";
00016
00017 constexpr char MENU_TEXT[] =
00018     "\nPasirinkite, ką norite daryti:\n"
00019     "1 - Vykdyti programą\n"
00020     "2 - Atlikti tyrimą\n"
00021     "0 - Nieko nenoriu daryti\n";
00022
00023 constexpr char TEST_MENU_TEXT[] =
00024     "\nPasirinkite tyrimą:\n"
00025     "1 - failu generavimas\n"
00026     "2 - programos veikimo laikas\n"
00027     "3 - Rule of Five testavimas\n"
00028     "0 - Užbaigti programą\n";
00029
00030 constexpr char PROGRAM_MENU_TEXT[] =
00031     "\nPasirinkite, ką norite daryti:\n"
00032     "1 - Įvesti viską rankiniu būdu\n"
00033     "2 - Generuoti atsitiktinius pažymius (vardus įvesti ranka)\n"
00034     "3 - Generuoti atsitiktinius vardus ir pažymius\n"
00035     "4 - Skaityti iš failo\n"
00036     "5 - Užbaigti programą\n";
00037
00038 constexpr char ENTER_FILE_SIZE[] =
00039     "\nPasirinkite failo dydį:\n"
00040     "1 - 1 000\n"
00041     "2 - 10 000\n"
00042     "3 - 100 000\n"
00043     "4 - 1 000 000\n"
00044     "5 - 10 000 000\n";
00045
00046 // errors
00047 const string INVALID_FIRST_NAME_ERROR = "\nKlaida: įvestas netinkamas vardas! ";
00048 const string INVALID_LAST_NAME_ERROR = "\nKlaida: įvesta netinkama pavardė! ";
00049 const string INVALID_MARK_ERROR = "\nKlaida: pažymys turi būti sveikasis skaičius nuo 1 iki 10! ";
00050 const string INVALID_EXAM_MARK_ERROR = "\nKlaida: egzamino pažymys turi būti sveikasis skaičius nuo 1
iki 10! ";
00051 const string INVALID_MARKS_COUNT_ERROR = "\nKlaida: pažymių skaičius turi būti sveikasis skaičius ir
mažesnis nei 10! ";
```

```

00052 const string INVALID_MENU_CHOICE = "\nKlaida: netinkamas meniu pasirinkimas! ";
00053 const string INVALID_CHOICE = "\nKlaida: netinkamas pasirinkimas! ";
00054 const string INVALID_SIZE_ERROR = "\nKlaida: dydis turi būti sveikasis skaičius! ";
00055 const string INVALID_MARKS_COUNT = "\nKlaida: turite įvesti bent vieną pažymį! ";
00056 const string INVALID_OUTPUT_TYPE_ERROR = "\nKlaida: netinkamas išvedimo tipas! ";
00057 const string FILE_OPEN_ERROR = "\nKlaida: nepavyko atidaryti failo! ";
00058 const string FILE_READ_ERROR = "\nKlaida: nepavyko nuskaityti failo! ";
00059 const string EMPTY_ARRAY_ERROR = "\nKlaida: pažymių masyvas tuščias! ";
00060 const string INVALID_GROUP_TYPE = "\nKlaida: netinkamas grupavimo tipas! ";
00061 const string INVALID_FILE_SIZE = "\nKlaida: netinkamas failo dydis! ";
00062
00063 // success
00064 const string FILE_READ_SUCCESS = "\nDuomenys nuskaityti sėkmingai! ";
00065 const string FILE_WRITE_SUCCESS = "\nDuomenys įrašyti sėkmingai! ";
00066 const string TEST_SECEESS = "\nLaiko tyrimo rezultatai įrašyti į failą: ";

```

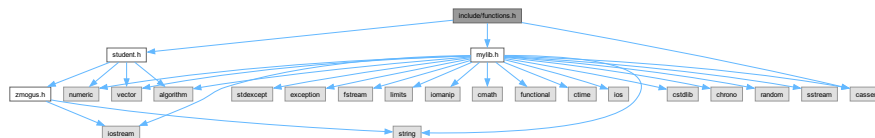
6.3 include/functions.h Failo Nuoroda

```

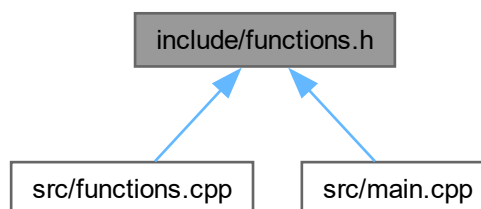
#include "mylib.h"
#include "student.h"
#include <cassert>

```

Įtraukimo priklausomybių diagrama functions.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

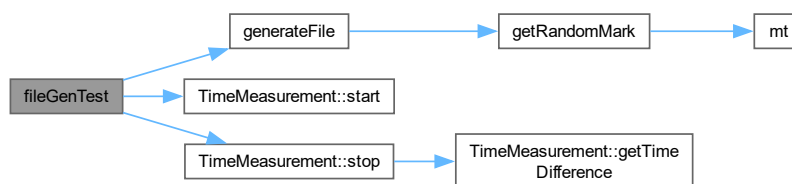
- void `handleProgramMenu` (vector< `Student` > &students)
- void `handleTestMenu` ()
- void `generateFile` (int size)
- void `groupStudents` (vector< `Student` > &students, vector< `Student` > &kietiakai, vector< `Student` > &vargsiukai, int groupType)
- void `printToConsole` (vector< `Student` > &kietiakai, vector< `Student` > &vargsiukai)
- void `printToFile` (vector< `Student` > &students, const string &fileName)
- void `fileGenTest` (int size)
- void `programTest` (int size)
- void `testRuleOfFive` ()

6.3.1 Funkcijos Dokumentacija

6.3.1.1 fileGenTest()

```
void fileGenTest (  
    int size)
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.3.1.2 generateFile()

```
void generateFile (  
    int size)
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:

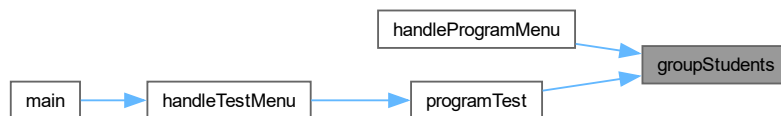


6.3.1.3 groupStudents()

```

void groupStudents (
    vector< Student > & students,
    vector< Student > & kietiakai,
    vector< Student > & vargsiukai,
    int groupType)
  
```

Here is the caller graph for this function:

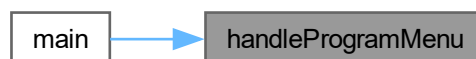


6.3.1.4 handleProgramMenu()

```

void handleProgramMenu (
    vector< Student > & students)
  
```

Here is the caller graph for this function:



6.3.1.5 handleTestMenu()

```
void handleTestMenu ()
```

Funkcijos kvietimo grafas:



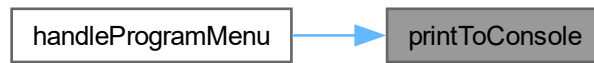
Here is the caller graph for this function:



6.3.1.6 printToConsole()

```
void printToConsole (
    vector< Student > & kietiakai,
    vector< Student > & vargsiukai)
```


Here is the caller graph for this function:

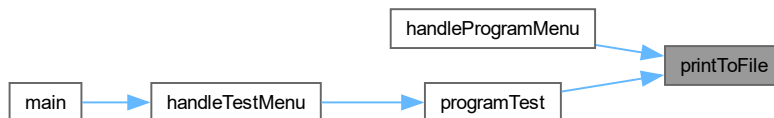


6.3.1.7 printToFile()

```

void printToFile (
    vector< Student > & students,
    const string & fileName)
  
```

Here is the caller graph for this function:

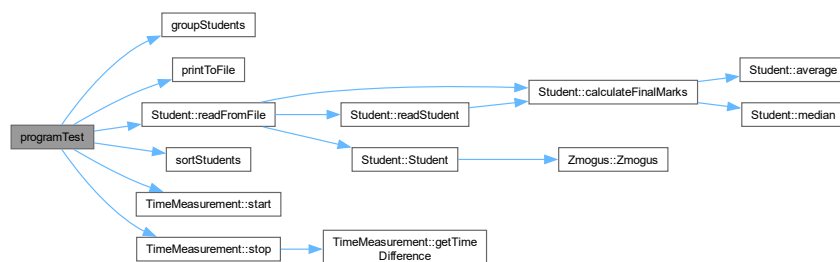


6.3.1.8 programTest()

```

void programTest (
    int size)
  
```

Funkcijos kvietimo grafas:



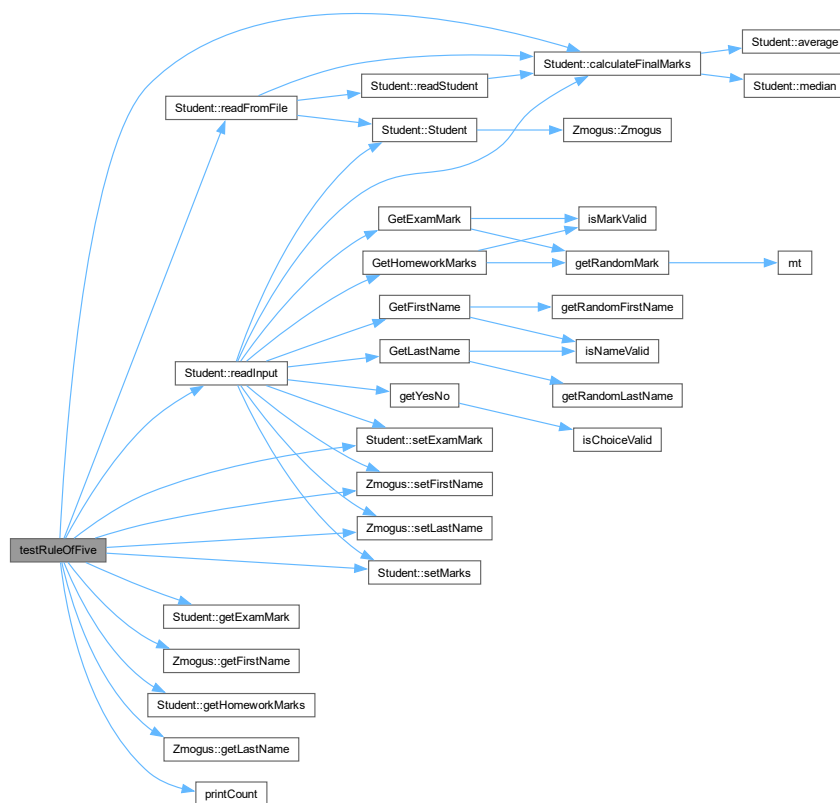
Here is the caller graph for this function:



6.3.1.9 testRuleOfFive()

```
void testRuleOfFive ()
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.4 functions.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #pragma once
00002 #include "mylib.h"
00003 #include "student.h"
00004 #include <cassert>
00005
00006 void handleProgramMenu(vector<Student>& students);
00007 void handleTestMenu();
00008 void generateFile(int size);
00009 void groupStudents(vector<Student>& students, vector<Student>& kietiakai, vector<Student>& vargsiukai,
00010 int groupType);
00011 void printToConsole(vector<Student>& kietiakai, vector<Student>& vargsiukai);
00012 void printToFile(vector<Student>& students, const string& fileName);
00013 void fileGenTest(int size);
00014 void programTest(int size);
00015 void testRuleOfFive();
00016 //void testZmogausClass();
  
```

6.5 include/mylib.h Failo Nuoroda

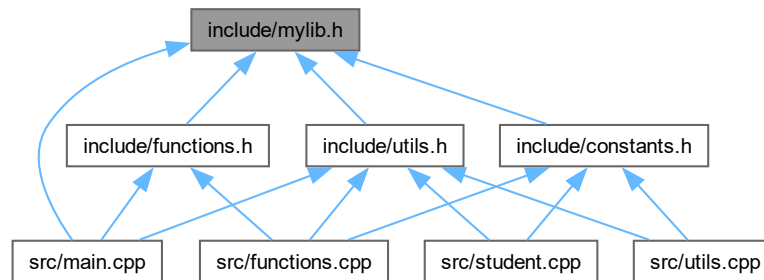
```

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <limits>
#include <iomanip>
#include <cmath>
#include <functional>
#include <algorithm>
#include <numeric>
#include <ctime>
#include <ios>
#include <cstdlib>
#include <chrono>
#include <random>
#include <sstream>
#include <cassert>
#include <stdexcept>
#include <exception>
  
```

Įtraukimo priklausomybių diagrama mylib.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



6.6 mylib.h

Eiti į šio failo dokumentaciją.

```

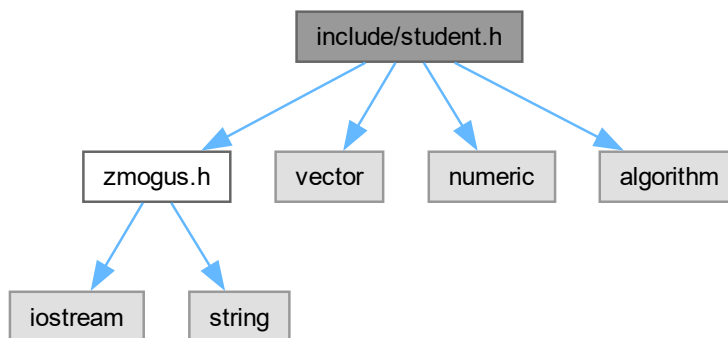
00001 #pragma once
00002 #include <iostream>
00003 #include <fstream>
00004 #include <string>
00005 #include <vector>
00006 #include <limits>
00007 #include <iomanip>
00008 #include <cmath>
00009 #include <functional>
00010 #include <algorithm>
00011 #include <numeric>
00012 #include <ctime>
00013 #include <limits>
00014 #include <ios>
00015 #include <cstdlib>
00016 #include <chrono>
00017 #include <random>
00018 #include <sstream>
00019 #include <cassert>
00020 #include <stdexcept>
00021 #include <exception>
00022
00023
00024 using std::cin;
00025 using std::cout;
00026 using std::endl;
00027 using std::string;
00028 using std::vector;
00029 using std::setw;
00030 using std::left;
00031 using std::right;
00032 using std::fixed;
00033 using std::setprecision;
00034 using std::numeric_limits;
00035 using std::streamsize;
00036 using std::sort;
00037 using std::stoi;
00038 using std::move;
00039 using std::ostringstream;
00040 using std::istringstream;
00041 using std::ifstream;
00042 using std::ofstream;
00043 using std::istream;
00044 using std::ostream;
00045 using std::cerr;
00046 using std::to_string;

```

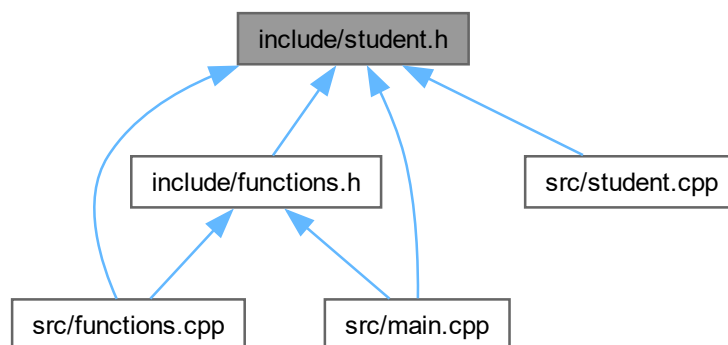
6.7 include/student.h Failo Nuoroda

```
#include "zmogus.h"  
#include <vector>  
#include <numeric>  
#include <algorithm>
```

Įtraukimo priklausomybių diagrama student.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [Student](#)

Funkcijos

- void [sortStudents](#) (std::vector< [Student](#) > &students, char sortType)

Kintamieji

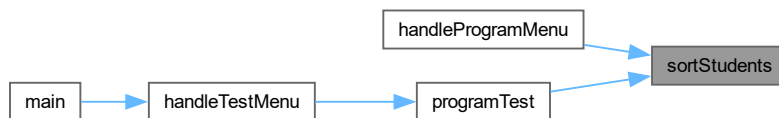
- int `dstCount`

6.7.1 Funkcijos Dokumentacija

6.7.1.1 sortStudents()

```
void sortStudents (
    std::vector< Student > & students,
    char sortType)
```

Here is the caller graph for this function:



6.7.2 Kintamojo Dokumentacija

6.7.2.1 dstCount

```
int dstCount [extern]
```

6.8 student.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002 #include "zmogus.h"
00003 #include <vector>
00004 #include <numeric>
00005 #include <algorithm>
00006
00007 extern int dstCount; // for destructor test
00008
00009 class Student : public Zmogus {
00010 private:
00011     std::vector<int> marks_;
00012     int examMark_;
00013     float avgFinal_;
00014     float medianFinal_;
00015
00016     // private methods
00017     float average(const std::vector<int>& marks) const;
00018     float median(const std::vector<int>& marks) const;
00019
00020 public:
00021     // constructors
00022     Student();
00023     Student(std::istream& is);
00024     /*Student(const std::string& firstName, const std::string& lastName, const std::vector<int>&
marks, int examMark)
00025         : Zmogus(firstName, lastName), marks_(marks), examMark_(examMark) {
```

```

00026         calculateFinalMarks();
00027     }*/
00028
00029     // rule of five
00030     Student(const Student& other); // copy constructor
00031     Student& operator=(const Student& other); // copy assignment operator
00032     Student(Student&& other) noexcept; // move constructor
00033     Student& operator=(Student&& other) noexcept; // move assignment operator
00034     ~Student(); // destructor
00035
00036     // getters
00037     const std::vector<int>& getHomeworkMarks() const { return marks_; }
00038     int getExamMark() const { return examMark_; }
00039     float getAvgFinal() const { return avgFinal_; }
00040     float getMedianFinal() const { return medianFinal_; }
00041
00042     // setters
00043     void setMarks(std::vector<int> marks) { marks_ = marks; }
00044     void setExamMark(int examMark) { examMark_ = examMark; }
00045
00046     // implementation of virtual functions
00047     void print(std::ostream& os) const override;
00048     void read(std::istream& is) override;
00049
00050     // methods
00051     std::istream& readStudent(std::istream& is);
00052     void calculateFinalMarks();
00053     static void readInput(std::vector<Student>& students, char menuChoice);
00054     static void readFromFile(std::vector<Student>& students, int fileSize);
00055 };
00056
00057 void sortStudents(std::vector<Student>& students, char sortType);

```

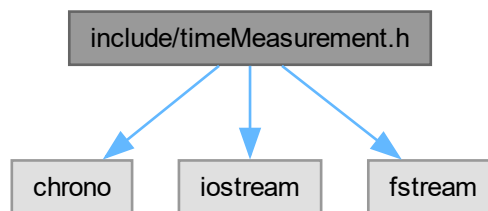
6.9 include/timeMeasurement.h Failo Nuoroda

```

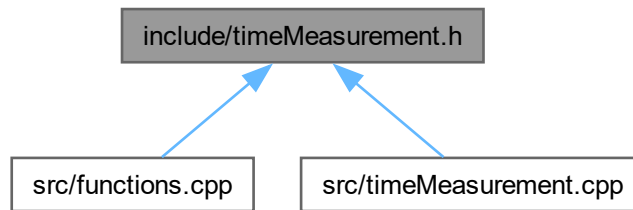
#include <chrono>
#include <iostream>
#include <fstream>

```

Įtraukimo priklausomybių diagrama timeMeasurement.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [TimeMeasurement](#)

6.10 timeMeasurement.h

Eiti į šio failo dokumentaciją.

```

00001 #pragma once
00002 #include <chrono>
00003 #include <iostream>
00004 #include <fstream>
00005
00006 class TimeMeasurement {
00007 private:
00008     std::chrono::high_resolution_clock::time_point startTime;
00009     std::chrono::high_resolution_clock::time_point endTime;
00010     std::string actionName;
00011
00012 public:
00013     // Constructor with an action name
00014     TimeMeasurement(const std::string& name);
00015
00016     // Start measurement
00017     void start();
00018
00019     // Stop measurement and log the time to file
00020     void stop(std::ofstream& runTimeResults);
00021
00022     // Get time difference (seconds)
00023     double getTimeDifference();
00024
00025     // Getter for action name (optional)
00026     std::string getActionName() const { return actionName; }
00027 };
00028
  
```

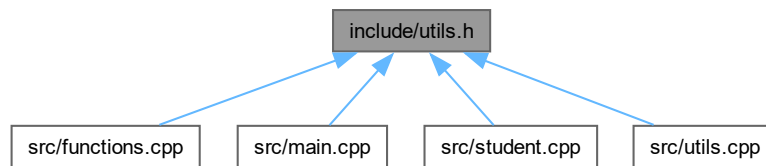
6.11 include/utils.h Failo Nuoroda

```
#include "mylib.h"
```

Įtraukimo priklausomybių diagrama utils.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

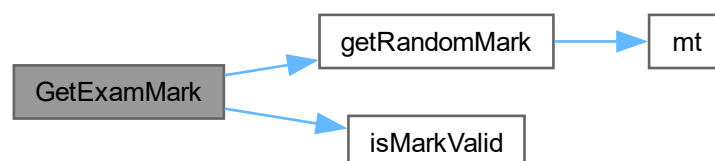
- string [getYesNo](#) ()
- int [getSortType](#) ()
- int [getGroupType](#) ()
- int [getPrintType](#) ()
- int [getMenuChoice](#) ()
- int [getProgramMenuChoice](#) ()
- int [getTestMenuChoice](#) ()
- int [getFileSize](#) ()
- string [GetFirstName](#) (char menuChoice)
- string [GetLastName](#) (char menuChoice)
- int [GetExamMark](#) (char menuChoice)
- vector< int > [GetHomeworkMarks](#) (char menuChoice)
- bool [isNameValid](#) (string name)
- bool [isMarkValid](#) (string mark)
- bool [isChoiceValid](#) (string choice)
- string [getRandomFirstName](#) ()
- string [getRandomLastName](#) ()
- int [getRandomMark](#) ()
- string [printCount](#) (int count)

6.11.1 Funkcijos Dokumentacija

6.11.1.1 GetExamMark()

```
int GetExamMark (
    char menuChoice)
```

Funkcijos kvietimo grafas:



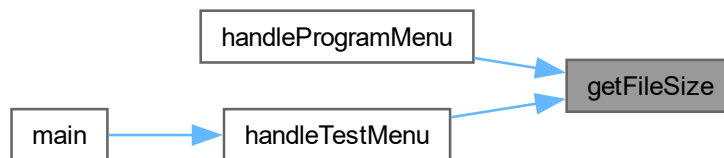
Here is the caller graph for this function:



6.11.1.2 `getFileSize()`

```
int getFileSize ()
```

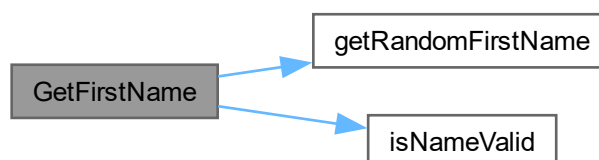
Here is the caller graph for this function:



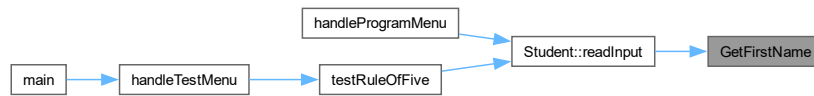
6.11.1.3 `GetFirstName()`

```
string GetFirstName (  
    char menuChoice)
```

Funkcijos kvietimo grafas:



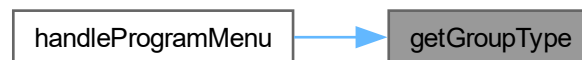
Here is the caller graph for this function:



6.11.1.4 getGroupType()

```
int getGroupType ()
```

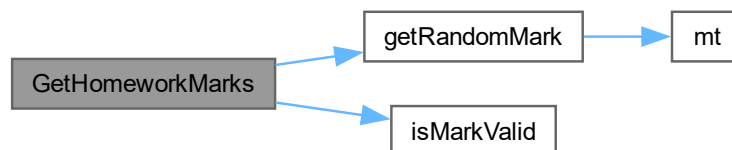
Here is the caller graph for this function:



6.11.1.5 GetHomeworkMarks()

```
vector< int > GetHomeworkMarks (
    char menuChoice)
```

Funkcijos kvietimo grafas:



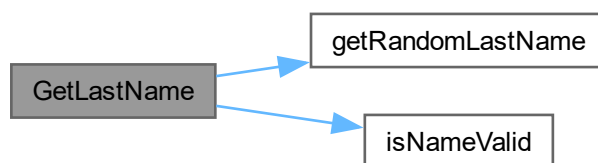
Here is the caller graph for this function:



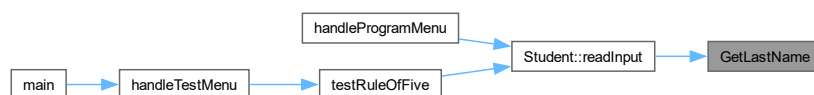
6.11.1.6 GetLastName()

```
string GetLastName (  
    char menuChoice)
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.11.1.7 getMenuChoice()

```
int getMenuChoice ()
```

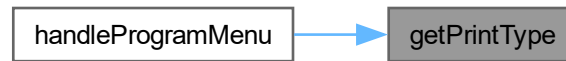
Here is the caller graph for this function:



6.11.1.8 getPrintType()

```
int getPrintType ()
```

Here is the caller graph for this function:



6.11.1.9 getProgramMenuChoice()

```
int getProgramMenuChoice ()
```

Here is the caller graph for this function:



6.11.1.10 getRandomFirstName()

```
string getRandomFirstName ()
```

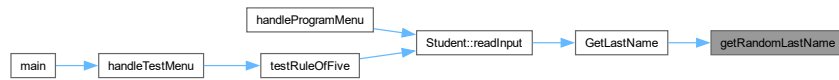
Here is the caller graph for this function:



6.11.1.11 getRandomLastName()

```
string getRandomLastName ()
```

Here is the caller graph for this function:



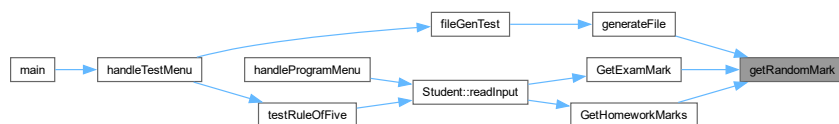
6.11.1.12 getRandomMark()

```
int getRandomMark ()
```

Funkcijos kvietimo grafas:



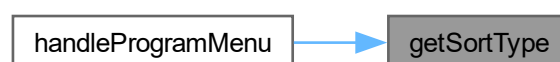
Here is the caller graph for this function:



6.11.1.13 getSortType()

```
int getSortType ()
```

Here is the caller graph for this function:



6.11.1.14 `getTestMenuChoice()`

```
int getTestMenuChoice ()
```

Here is the caller graph for this function:



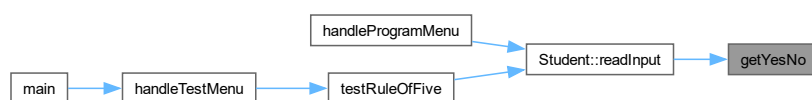
6.11.1.15 `getYesNo()`

```
string getYesNo ()
```

Funkcijos kvietimo grafas:



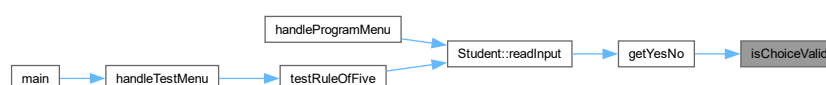
Here is the caller graph for this function:



6.11.1.16 `isChoiceValid()`

```
bool isChoiceValid (  
    string choice)
```

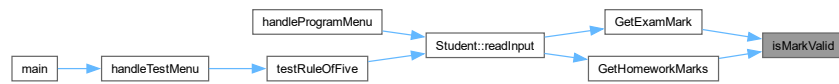
Here is the caller graph for this function:



6.11.1.17 isMarkValid()

```
bool isMarkValid (
    string mark)
```

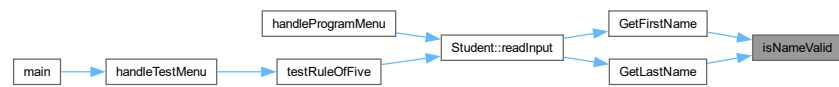
Here is the caller graph for this function:



6.11.1.18 isNameValid()

```
bool isNameValid (
    string name)
```

Here is the caller graph for this function:



6.11.1.19 printCount()

```
string printCount (
    int count)
```

Here is the caller graph for this function:



6.12 utils.h

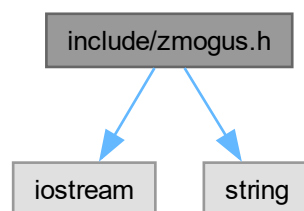
Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002 #include "mylib.h"
00003
00004 // requests
00005 string getYesNo();
00006 int getSortType();
00007 int getGroupType();
00008 int getPrintType();
00009 int getMenuChoice();
00010 int getProgramMenuChoice();
00011 int getTestMenuChoice();
00012 int getFileSize();
00013 string GetFirstName(char menuChoice);
00014 string GetLastName(char menuChoice);
00015 int GetExamMark(char menuChoice);
00016 vector<int> GetHomeworkMarks(char menuChoice);
00017
00018 // validations
00019 bool isNameValid(string name);
00020 bool isMarkValid(string mark);
00021 bool isChoiceValid(string choice);
00022
00023 // random generators
00024 string getRandomFirstName();
00025 string getRandomLastName();
00026 int getRandomMark();
00027
00028 // order track
00029 string printCount(int count);
```

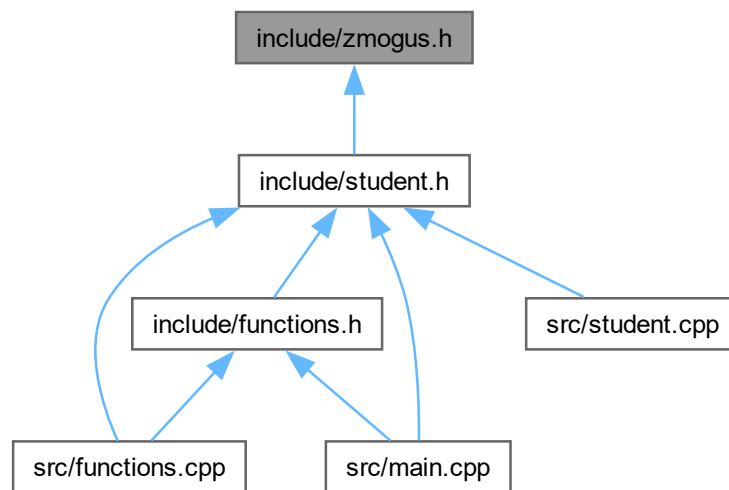
6.13 include/zmogus.h Failo Nuoroda

```
#include <iostream>
#include <string>
```

Įtraukimo priklausomybių diagrama zmogus.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [Zmogus](#)

Funkcijos

- `std::istream & operator>> (std::istream &is, Zmogus &zmogus)`
- `std::ostream & operator<< (std::ostream &os, const Zmogus &zmogus)`

6.13.1 Funkcijos Dokumentacija

6.13.1.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const Zmogus & zmogus) [inline]
```

Funkcijos kvietimo grafas:



6.13.1.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Zmogus & zmogus) [inline]
```

Funkcijos kvietimo grafas:



6.14 zmogus.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002 #include <iostream>
00003 #include <string>
00004
00005 class Zmogus {
00006 protected:
00007     std::string firstName_;
00008     std::string lastName_;
00009
00010 public:
00011     Zmogus() : firstName_(""), lastName_("") {}
00012     Zmogus(const std::string& firstName, const std::string& lastName)
00013         : firstName_(firstName), lastName_(lastName) {}
00014
00015     virtual ~Zmogus() = default; // virtual destructor
00016
00017     // abstract class interface
00018     virtual void read(std::istream& is) = 0;
00019     virtual void print(std::ostream& os) const = 0;
00020
00021     // getters/setters
00022     std::string getFirstName() const { return firstName_; }
00023     std::string getLastName() const { return lastName_; }
00024     void setFirstName(const std::string& firstName) { firstName_ = firstName; }
00025     void setLastName(const std::string& lastName) { lastName_ = lastName; }
00026 };
00027
00028 // global operators
00029 inline std::istream& operator>>(std::istream& is, Zmogus& zmogus) {
00030     zmogus.read(is);
00031     return is;
00032 }
00033
00034 inline std::ostream& operator<<(std::ostream& os, const Zmogus& zmogus) {
00035     zmogus.print(os);
00036     return os;
00037 }
```

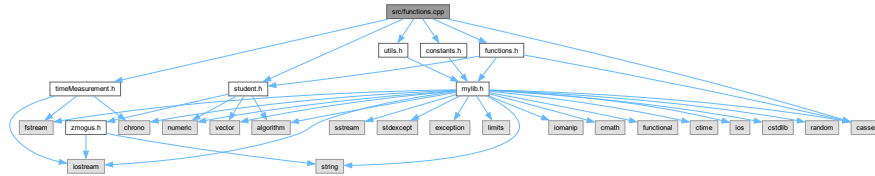
6.15 README.md Failo Nuoroda

6.16 src/functions.cpp Failo Nuoroda

```
#include "functions.h"
#include "utils.h"
```

```
#include "constants.h"
#include "timeMeasurement.h"
#include "student.h"
#include <cassert>
```

Įtraukimo priklausomybių diagrama functions.cpp:



Funkcijos

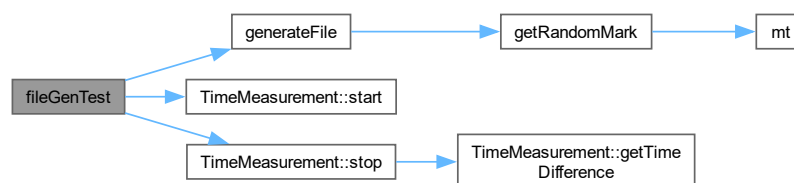
- void `handleProgramMenu` (std::vector< `Student` > &students)
- void `handleTestMenu` ()
- void `generateFile` (int size)
- void `groupStudents` (vector< `Student` > &students, vector< `Student` > &kietiakai, vector< `Student` > &vargsiukai, int groupType)
- void `printToConsole` (vector< `Student` > &kietiakai, vector< `Student` > &vargsiukai)
- void `printToFile` (vector< `Student` > &students, const string &fileName)
- void `fileGenTest` (int size)
- void `programTest` (int size)
- void `testRuleOfFive` ()

6.16.1 Funkcijos Dokumentacija

6.16.1.1 fileGenTest()

```
void fileGenTest (
    int size)
```

Funkcijos kvietimo grafas:



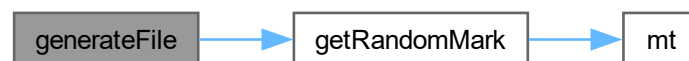
Here is the caller graph for this function:



6.16.1.2 generateFile()

```
void generateFile (  
    int size)
```

Funkcijos kvietimo grafas:



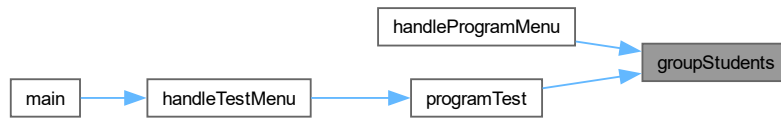
Here is the caller graph for this function:



6.16.1.3 groupStudents()

```
void groupStudents (  
    vector< Student > & students,  
    vector< Student > & kietiakai,  
    vector< Student > & vargsiukai,  
    int groupType)
```

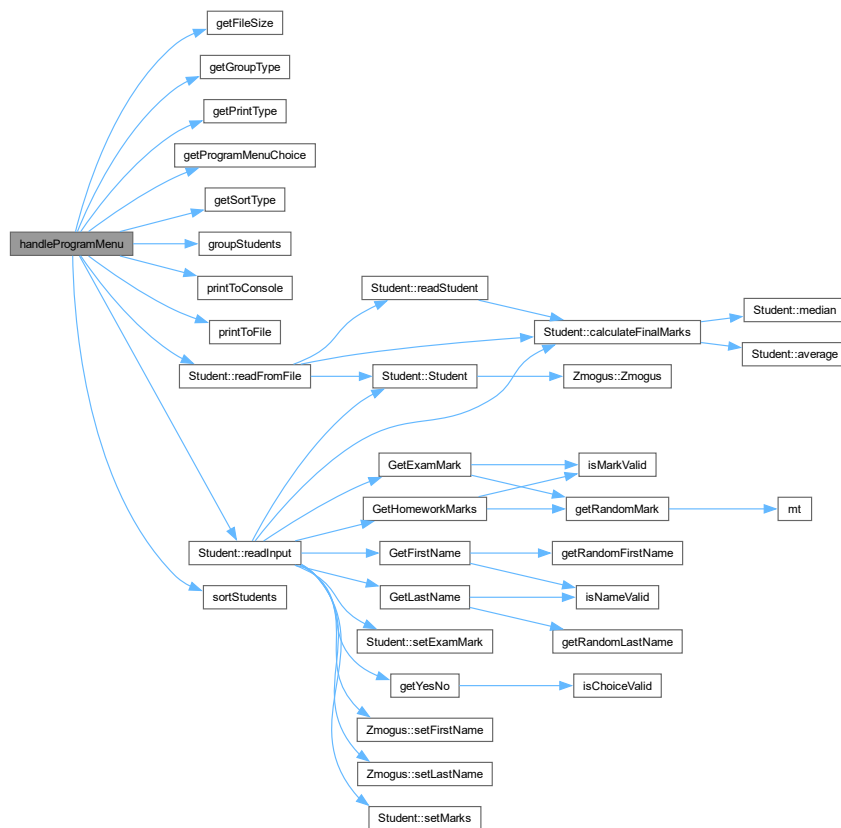
Here is the caller graph for this function:



6.16.1.4 handleProgramMenu()

```
void handleProgramMenu (
    std::vector< Student > & students)
```

Funkcijos kvietimo grafas:



6.16.1.5 handleTestMenu()

```
void handleTestMenu ()
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.16.1.6 printToConsole()

```
void printToConsole (
    vector< Student > & kietiakai,
    vector< Student > & vargsiukai)
```

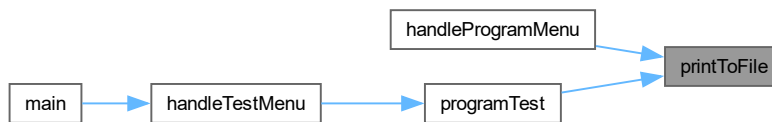
Here is the caller graph for this function:



6.16.1.7 printToFile()

```
void printToFile (
    vector< Student > & students,
    const string & fileName)
```

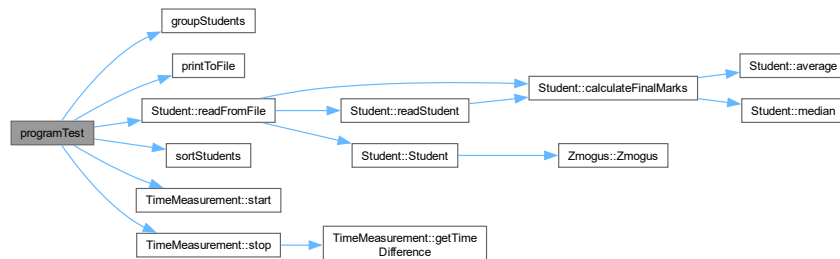
Here is the caller graph for this function:



6.16.1.8 programTest()

```
void programTest (
    int size)
```

Funkcijos kvietimo grafas:



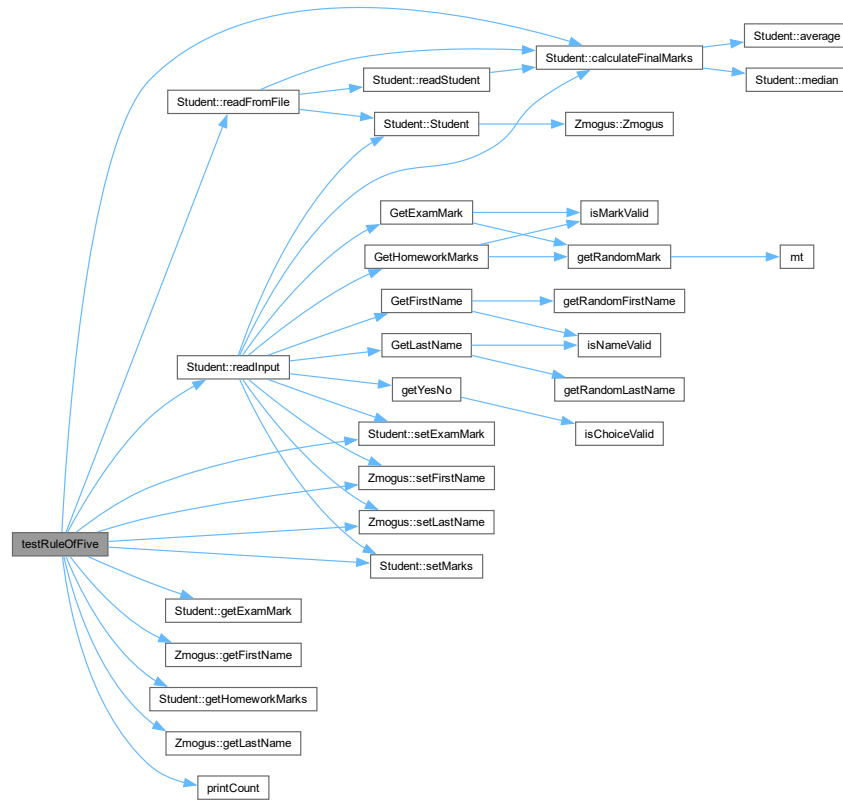
Here is the caller graph for this function:



6.16.1.9 testRuleOfFive()

```
void testRuleOfFive ()
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



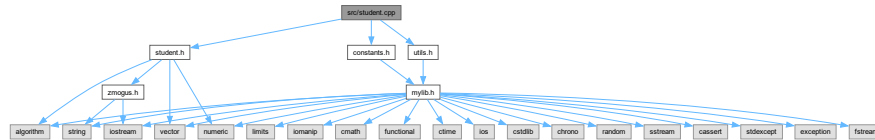
6.17 src/main.cpp Failo Nuoroda

```
#include "mylib.h"
#include "utils.h"
#include "functions.h"
```


6.18 src/student.cpp Failo Nuoroda

```
#include "student.h"
#include "utils.h"
#include "constants.h"
```

Įtraukimo priklausomybių diagrama student.cpp:



Funkcijos

- void `sortStudents` (std::vector< `Student` > &students, char sortType)

Kintamieji

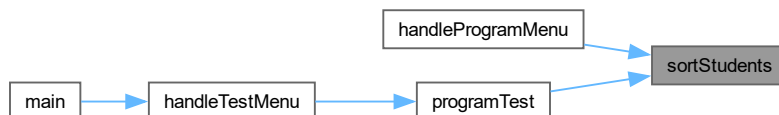
- int `dstCount` = 0

6.18.1 Funkcijos Dokumentacija

6.18.1.1 sortStudents()

```
void sortStudents (
    std::vector< Student > & students,
    char sortType)
```

Here is the caller graph for this function:



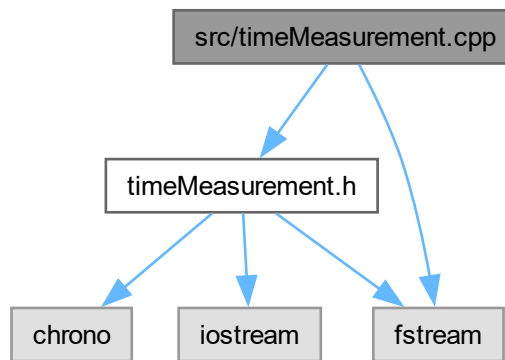
6.18.2 Kintamojo Dokumentacija

6.18.2.1 dstCount

```
int dstCount = 0
```

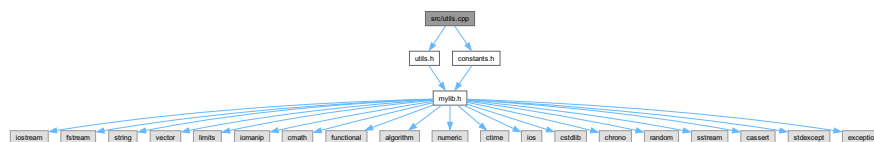
6.19 src/timeMeasurement.cpp Failo Nuoroda

```
#include "timeMeasurement.h"
#include <fstream>
Įtraukimo priklausomybių diagrama timeMeasurement.cpp:
```



6.20 src/utlis.cpp Failo Nuoroda

```
#include "utlis.h"
#include "constants.h"
Įtraukimo priklausomybių diagrama utlis.cpp:
```



Funkcijos

- `std::mt19937 mt(rd())`
- `string getYesNo()`
- `int getSortType()`
- `int getPrintType()`
- `int getMenuChoice()`
- `int getTestMenuChoice()`
- `int getProgramMenuChoice()`
- `string GetFirstName(char menuChoice)`
- `string GetLastName(char menuChoice)`
- `int GetExamMark(char menuChoice)`
- `vector< int > GetHomeworkMarks(char menuChoice)`

- int `getGroupType` ()
- int `getFileSize` ()
- bool `isNameValid` (string name)
- bool `isMarkValid` (string mark)
- bool `isChoiceValid` (string choice)
- string `getRandomFirstName` ()
- string `getRandomLastName` ()
- int `getRandomMark` ()
- string `printCount` (int count)

Kintamieji

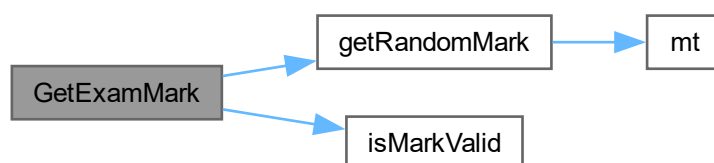
- `std::random_device rd`

6.20.1 Funkcijos Dokumentacija

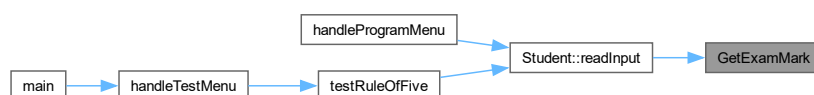
6.20.1.1 GetExamMark()

```
int GetExamMark (  
    char menuChoice)
```

Funkcijos kvietimo grafas:



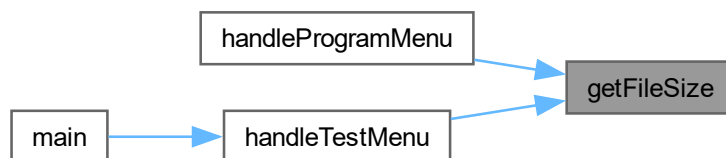
Here is the caller graph for this function:



6.20.1.2 getFileSize()

```
int getFileSize ()
```

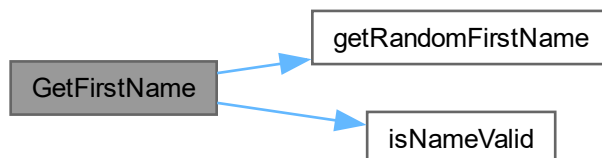
Here is the caller graph for this function:



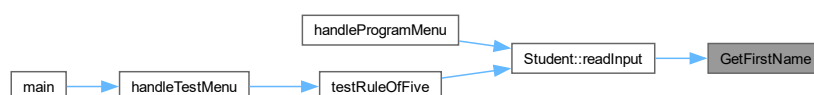
6.20.1.3 GetFirstName()

```
string GetFirstName (  
    char menuChoice)
```

Funkcijos kvietimo grafas:



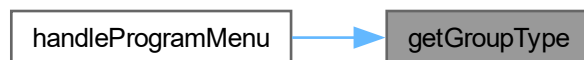
Here is the caller graph for this function:



6.20.1.4 getGroupType()

```
int getGroupType ()
```

Here is the caller graph for this function:



6.20.1.5 GetHomeworkMarks()

```
vector< int > GetHomeworkMarks (  
    char menuChoice)
```

Funkcijos kvietimo grafas:



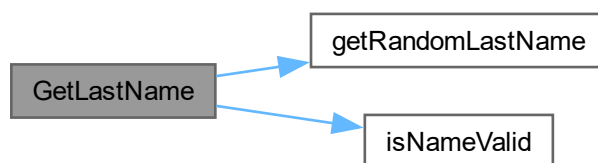
Here is the caller graph for this function:



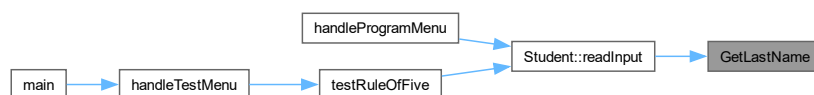
6.20.1.6 GetLastName()

```
string GetLastName (  
    char menuChoice)
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.20.1.7 getMenuChoice()

```
int getMenuChoice ()
```

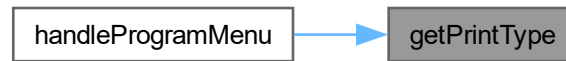
Here is the caller graph for this function:



6.20.1.8 getPrintType()

```
int getPrintType ()
```

Here is the caller graph for this function:



6.20.1.9 getProgramMenuChoice()

```
int getProgramMenuChoice ()
```

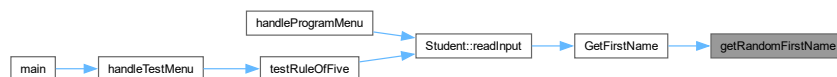
Here is the caller graph for this function:



6.20.1.10 getRandomFirstName()

```
string getRandomFirstName ()
```

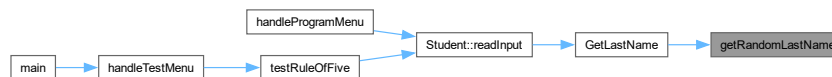
Here is the caller graph for this function:



6.20.1.11 getRandomLastName()

```
string getRandomLastName ()
```

Here is the caller graph for this function:



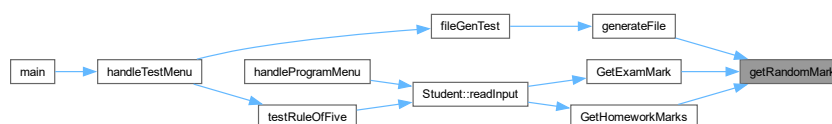
6.20.1.12 getRandomMark()

```
int getRandomMark ()
```

Funkcijos kvietimo grafas:



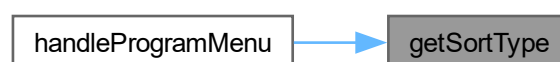
Here is the caller graph for this function:



6.20.1.13 getSortType()

```
int getSortType ()
```

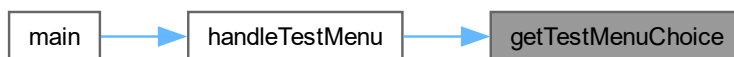
Here is the caller graph for this function:



6.20.1.14 `getTestMenuChoice()`

```
int getTestMenuChoice ()
```

Here is the caller graph for this function:



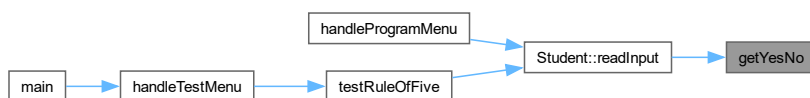
6.20.1.15 `getYesNo()`

```
string getYesNo ()
```

Funkcijos kvietimo grafas:



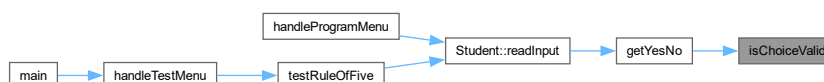
Here is the caller graph for this function:



6.20.1.16 `isChoiceValid()`

```
bool isChoiceValid (  
    string choice)
```

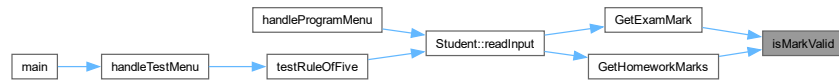
Here is the caller graph for this function:



6.20.1.17 isMarkValid()

```
bool isMarkValid (
    string mark)
```

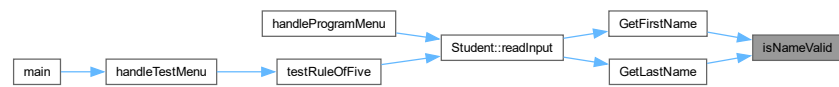
Here is the caller graph for this function:



6.20.1.18 isNameValid()

```
bool isNameValid (
    string name)
```

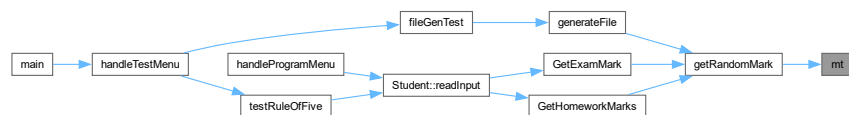
Here is the caller graph for this function:



6.20.1.19 mt()

```
std::mt19937 mt (
    rd() )
```

Here is the caller graph for this function:



6.20.1.20 printCount()

```
string printCount (  
    int count)
```

Here is the caller graph for this function:



6.20.2 Kintamojo Dokumentacija

6.20.2.1 rd

```
std::random_device rd
```

